Claude Transcript
EvalGen Implementation

Companion implementation conversation, reproduced from the repository transcript. Speaker labels and source order are preserved.

Typographic dashes are normalized for printing; thumbs-up and thumbs-down symbols are spelled out in brackets.

User

here's another job, for the 2nd paper (evalgen)

https://github.com/EvalGen/EvalGen

is the source repo

i want you to clone it to tmp, then re-implement it in ./hackathon with /kuroma-layout and a config.toml that points to the LLM at localhost:11434, same styling (colors and themes and fonts) as /home/kuroma/Documents/projects/personal/kkuroma.github.io

Tool calls

- Skill: kuroma-layout
- Bash: Clone EvalGen repo to /tmp and list files
- Bash: Locate hackathon dir and list the style source site
- Bash: Inspect existing hackathon directory
- Bash: Read EvalGen source except the streamlit app
- Bash: Read EvalGen streamlit app and configs
- Bash: Read the site's themes, index page and CLAUDE.md
- Bash: Read the site's standalone-page theme boot and CSS

Claude

I'm reading the site's CSS so the app's boxes, buttons and fonts match it.

Tool calls

- Bash: Read site CSS variables, fonts and key component rules
- Bash: Read remaining style rules, settings menu code, and check Ollama
- Bash: Probe local LLM server and read settings menu code
- Bash: List local models and find EvalGen metric definitions
- Bash: Read the EvalGen paper's alignment definitions

- Bash: Check upstream license and hackathon git state
- Bash: Create project dirs and copy site theme and fonts
- Write: `hackathon/pyproject.toml`
- Write: `hackathon/config.toml`
- Write: `hackathon/.gitignore`
- Write: `hackathon/.env.example`
- Write: `hackathon/src/evalgen/__init__.py`
- Write: `hackathon/src/evalgen/config.py`
- Write: `hackathon/src/evalgen/prompts/__init__.py`
- Write: `hackathon/src/evalgen/prompts/generate_criteria.jinja`
- Write: `hackathon/src/evalgen/prompts/generate_assertions.jinja`
- Write: `hackathon/src/evalgen/prompts/evaluate.jinja`
- Write: `hackathon/src/evalgen/llm.py`

User

[Request interrupted by user]

User

run init-shell inside of that dir to set up nix flake too

Tool calls

- Bash: Find and read the init-shell command
- Bash: Read the rest of init-shell

Claude

`init-shell --python` runs `uv init`, which refuses to start when a `pyproject.toml` exists. I'll move mine aside, run it, then restore mine over the generated one.

Tool calls

- Bash: Run init-shell to create the Nix flake and venv
- Bash: Inspect files generated by init-shell
- Bash: Remove generated stub package and restore my pyproject
- Edit: `hackathon/pyproject.toml`
- Edit: `hackathon/pyproject.toml`
- Edit: `hackathon/pyproject.toml`

- Edit: `hackathon/pyproject.toml`
- Write: `hackathon/src/evalgen/llm.py`
- Write: `hackathon/src/evalgen/criteria.py`
- Write: `hackathon/src/evalgen/assertions.py`
- Write: `hackathon/src/evalgen/grading.py`
- Write: `hackathon/src/evalgen/alignment.py`
- Write: `hackathon/src/evalgen/app.py`
- Write: `hackathon/scripts/serve.py`

Claude

Now the page. It reuses the site's palette tokens, fonts, pill navbar, Aa appearance menu, and box and footer styles.

Tool calls

- Write: `hackathon/src/evalgen/web/index.html`
- Write: `hackathon/src/evalgen/web/appearance.js`
- Write: `hackathon/src/evalgen/web/app.css`
- Write: `hackathon/src/evalgen/web/app.js`

Claude

Next, tests and the directory READMEs.

Tool calls

- Write: `hackathon/tests/conftest.py`
- Write: `hackathon/tests/test_llm.py`
- Write: `hackathon/tests/test_prompts.py`
- Write: `hackathon/tests/test_alignment.py`
- Write: `hackathon/tests/test_stages.py`
- Write: `hackathon/tests/test_app.py`
- Write: `hackathon/tests/integration/test_live_llm.py`
- Bash: Install dependencies and run the test suite
- Bash: Run ruff, black, pyright and import-linter
- Bash: Fix type, lint and format issues, then rerun checks

Claude

Static checks and unit tests pass. Next, the live test against GPT-OSS-20B on :11434, then a real end-to-end run through the served app.

Tool calls

- Bash: Run the live LLM integration test
- Bash: Launch the EvalGen web app on port 8269
- Bash: Wait for the app, then request criteria from the live model
- Write: `scratchpad/e2e.py`
- Bash: Run the end-to-end flow against the live app
- Bash: Syntax-check the page scripts and confirm static files serve
- Bash: Screenshot the served page with headless Chromium
- Read: `scratchpad/page.png`

Claude

The page renders with the site's look. The one defect is that the empty toast peeks out as a red pill at the bottom edge. Fixing that, then the READMEs.

Tool calls

- Edit: `hackathon/src/evalgen/web/app.css`
- Write: `hackathon/README.md`

User

[Request interrupted by user]

User

do not git add or git commit anything, and be very concise with readmes too

Claude

The `init-shell --git` run staged files with `git add .`, so I'm unstaging everything. Your working tree stays as it is and nothing gets committed.

Tool calls

- Bash: Unstage everything init-shell added to the index
- Bash: Check what changed in README.md
- Read: `hackathon/README.md`

Claude

You've already written the root README, so I'm leaving it as is. The subdirectory READMEs are minimal, and the toast fix goes in too.

Tool calls

- Edit: `hackathon/src/evalgen/web/app.css`
- Write: `hackathon/src/README.md`
- Write: `hackathon/scripts/README.md`
- Write: `hackathon/tests/README.md`
- Bash: Confirm nothing is staged

Claude

EvalGen is rebuilt in `hackathon/` following kuroma-layout, and it runs end to end against GPT-OSS-20B on `localhost:11434`. Nothing is staged or committed: `init-shell --git` had run `git add .`, and I unstaged it all with `git reset` without touching your files.

What's there

- **Nix and Python:** `init-shell --python --git` created `flake.nix`. I replaced the `pyproject.toml` it generated with mine (uv build backend, your author line, import-linter contract, ruff/black/pyright config) and deleted the `src/hackathon` stub it made.
- `config.toml`: points at `http://localhost:11434/v1`, which is a llama.cpp server, not Ollama. It also sets the model, temperatures, concurrency, port 8269 and the FFR threshold. A key, if ever needed, goes in `.env`.
- `src/evalgen/`: one module per stage: config, prompts, llm, criteria, assertions, grading, alignment, app.
- `scripts/serve.py`: starts the app.
- `tests/`: 25 unit tests using a fake LLM, plus one live-server test marked `slow`.
- **Styling:** the site's `themes.js` copied as is, its Google Sans Flex and Maple Mono fonts, and its navbar, boxes, buttons and Aa menu (4 palettes, light/dark/auto, text size).

Compared with the upstream repo

1. Upstream's grader never saw the assertion, because the code passed `criteria=` to a template expecting `assertion`. The templates now raise an error on any missing variable.

2. Upstream stopped after running the assertions. I added the paper's §4.2 coverage, false failure rate (FFR) and alignment scores, picking one assertion per criterion under an FFR threshold, and a report card. Changing a grade or the threshold updates the report without new model calls.

3. Two parts of the paper are not implemented:

- Code-based assertions, since running model-written Python needs a sandbox.

- The grading sampler that picks which output to grade next; all outputs are shown at once.

4. Streamlit is replaced by FastAPI serving a static page, since Streamlit can't carry the site's palette and mode switching.

Checks: ruff, black, pyright, import-linter and the unit tests all pass, and so does the live test. In a full run with real criteria, assertions, samples and grades, the evaluate call took 81s for 24 grader calls.

Untested: a headless Chromium screenshot shows the empty page styled correctly. I have not seen the filled-in report table in a browser; its data came through the API correctly.

The app is still running at http://127.0.0.1:8269, and your root README is unchanged.

User

design the UI to reflect the version on the paper but using my colors that's all

Tool calls

- Bash: List the paper's figure captions
- Bash: Read figure captions and render early paper pages
- Bash: Find which paper pages hold figures
- Read: `scratchpad/paper/p-05.png`
- Bash: Crop Figure 2 panels at high resolution

Claude

I'm studying the paper's Figure 2 screens (Pick Criteria, Grade, Report Card) to copy their layout.

Tool calls

- Read: `scratchpad/paper/c-05.png`
- Read: `scratchpad/paper/d-05.png`
- Read: `scratchpad/paper/e-05.png`
- Read: `hackathon/src/evalgen/alignment.py`
- Bash: Add confusion counts and active criteria to alignment
- Write: `hackathon/src/evalgen/prompts/generate_criteria.jinja`
- Write: `hackathon/src/evalgen/criteria.py`
- Bash: Update API and tests for titled criteria, then run all checks

Claude

`nix develop` can't see an untracked `flake.nix`, and you've asked me not to `git add`, so I'll run the tools outside the flake.

Tool calls

- Bash: Run lint, types, tests and import contract without the flake
- Bash: Show the pyright error
- Bash: Fix the live test's return annotation and recheck types

Claude

Next, the new page: the canvas (Prompt node wired to Multi-Evaluator, plus the Table View) and the wizard dialog.

Tool calls

- Write: `hackathon/src/evalgen/web/index.html`
- Write: `hackathon/src/evalgen/web/icons.js`
- Write: `hackathon/src/evalgen/web/app.css`

Claude

Now the page logic. Assertion generation and execution run in the background while you grade, which is the paper's core design idea.

Tool calls

- Write: `hackathon/src/evalgen/web/app.js`

Claude

A background re-render while you're typing would reset a card's title. I'm fixing that, and moving the threshold refresh to slider release so dragging isn't interrupted.

Tool calls

- Bash: Keep typed text across re-renders and syntax-check

Claude

Restarting the server so it picks up the backend changes, then I'll drive the real flow in headless Chromium and screenshot each screen.

Tool calls

- ToolSearch: select:TaskStop
- TaskStop: bfepoquoe
- Bash: Relaunch the EvalGen web app on port 8269

Claude

I'll drive the real flow in headless Chromium with Playwright from nix, against the live model.

Tool calls

- Write: `scratchpad/drive.py`
- Bash: Run the browser walkthrough and capture screenshots

Claude

The sampling worked. The wait failed because the check mark span had no icon, so it rendered at zero size. Adding the icon and rerunning.

Tool calls

- Bash: Fix the check icon and rerun the browser walkthrough

Claude

Implementation didn't finish within 400 seconds, so I'm checking the server log and the grade screenshot.

Tool calls

- Bash: Show the app server's recent log

Claude

API Error: The response stopped arriving. The response above may be incomplete.

Tool calls

- Read: `/tmp/claude-1000/-home-kuroma-Documents-masters-year1-CS269-communicating-with-computers/ddf287f7-0213-4beb-9d52-6ecf18021c21/tasks/bxz4irf9h.output`

Claude

One `/api/evaluate` call never returned, so a single grader call is hanging upstream. First, the four screens that did render:

Tool calls

- Read: `scratchpad/shots/1-canvas.png`
- Read: `scratchpad/shots/2-start.png`
- Read: `scratchpad/shots/3-criteria.png`
- Read: `scratchpad/shots/4-grade.png`
- Bash: Check server log and llama.cpp slot state
- Bash: Cap grader reply length, fix title CSS, rerun checks

Claude

Restarting the server and rerunning the full browser walkthrough.

Tool calls

- TaskStop: bxz4irf9h
- Bash: Relaunch the EvalGen web app on port 8269
- Bash: Rerun the full browser walkthrough with screenshots

- Read: `/tmp/claude-1000/-home-kuroma-Documents-masters-year1-CS269-communicating-with-computers/ddf287f7-0213-4beb-9d52-6ecf18021c21/tasks/bc0g2mwoi.output`

Claude

One evaluate call hangs again, even with the token cap. I'm timing grader calls directly against :11434 to find where it stalls.

Tool calls

- Write: `scratchpad/probe.py`
- Bash: Time 12 concurrent grader calls against the local LLM

Claude

Direct calls are fast: 12 calls in 14 seconds, all ending normally. So the stall is inside the app. I'm checking whether the second evaluate eventually returned.

Tool calls

- Read: `/tmp/claude-1000/-home-kuroma-Documents-masters-year1-CS269-communicating-with-computers/ddf287f7-0213-4beb-9d52-6ecf18021c21/tasks/bc0g2mwoi.output`
- Bash: List open connections to the LLM server and app

Claude

The app holds four connections to :11434 in CLOSE-WAIT: the LLM server closed them, and the client is still waiting on them. I'm testing whether the app can make a fresh call right now.

Tool calls

- Bash: Send one small evaluate request to the app
- Bash: Rerun the walkthrough capturing browser console errors
- Bash: Show the walkthrough error and recent server log
- Read: `scratchpad/shots/5-report.png`
- Read: `scratchpad/shots/6-confusion.png`
- Read: `scratchpad/shots/7-table.png`
- Bash: Lower call timeout and screenshot the light Haruhana theme
- Read: `scratchpad/shots/9-light.png`

Claude

Light mode with the Haruhana palette works. Restarting the server so the new timeout takes effect.

Tool calls

- TaskStop: bc0g2mwoi

- Bash: Relaunch the EvalGen web app on port 8269
- Bash: Confirm the app is up and nothing is staged

Claude

The UI now follows the paper's Figure 2 and Figure 3 in your colors. I ran the full flow in headless Chromium against GPT-OSS-20B and every screen worked, but one earlier run stalled and I haven't confirmed the fix.

The screens map to the paper like this:

Paper	Here
2a: Prompt node wired to Multi-Evaluator	Canvas with a dot grid and a dashed wire between the two nodes. Run samples N responses; "Generate criteria" opens the wizard.
2b: Infer / Manual / Grade First	Three tinted option cards in your mauve, green and yellow. Grade First has the paper's tooltip.
2c: Pick Criteria	Text box where Enter adds a criterion, a blue "Suggest more" button, two-column cards (checkbox, editable title and description, trash, Code/LLM toggle), and the gradient "I'm done. Implement it!" button.
2d: Grade	"Is this response [thumbs up] or [thumbs down]?" with arrow navigation, a Feedback panel and a Prompt panel, Bad!/Good! buttons, and a progress bar with "I'm tired". Assertions are generated and run in the background while you grade.
2e: Report Card	Coverage and FFR tiles, a card per criterion with an alignment donut, a confusion matrix on hover, and "Show bad implementations".
3: Table View	Responses × kept criteria as pass/fail, plus your grade per response.

Supporting changes:

- **Criteria format:** criteria now come back as "Title: description", as on the paper's cards.
- **Grade First and Suggest more:** your grades and feedback go into the criteria prompt, as the paper describes.
- **Report data:** scores now include confusion counts for the hover matrix.
- **Tests:** 27 unit tests pass, and so do ruff, black, pyright and the import contract.

Two problems remain:

- **Intermittent stall.** On two of three runs, one grader call never returned and the grade screen sat at "Generating..." past 400 seconds. Called directly against :11434, the same kind of call takes 2 to 6 seconds. I capped grader replies at 2048 tokens, set the client to one retry, and lowered the per-call timeout to 120s. A stuck call should now fail over within about four minutes instead of hanging. The run that completed had the token cap but still the 300s timeout; I haven't rerun the flow since lowering it, so I can't confirm the stall is fixed.
- `nix develop` **and direnv fail** because `flake.nix` isn't tracked, and Nix only sees tracked files. They'll work once you run `git add flake.nix flake.lock`, or use `nix develop path:.` meanwhile. I ran the checks outside the flake rather than stage anything; nothing is staged or committed.

The Code toggle on the criteria cards is visible but disabled, since running model-written Python would need a sandbox.

The app is running at http://127.0.0.1:8269.

User

one last change: make it so the webui allows for model selection instead of hard coding gpt oss 20b

Claude

The prompt node and the Multi-Evaluator will each get their own model picker, filled from the server's `/v1/models`. The sampling model and the grading model can then differ, like the paper's split between pipeline model and evaluator model, and the prompt node's "Models to query" in Fig. 2a. `config.toml` keeps the default.

Tool calls

- Bash: Show the code regions to change
- Bash: Add per-request model choice to backend, tests, page markup and styles

Claude

Now the page logic: each picker is persisted, each call sends its node's model, and each criterion remembers which model built it.

Tool calls

- Bash: Wire model pickers in the page and run all checks

Claude

All checks pass (the line-length error was fixed by black's reformat). Restarting the server, then checking that model listing and a non-default model work live.

Tool calls

- TaskStop: b2z4ouekt
- Bash: Relaunch the EvalGen web app on port 8269
- Bash: List models, call a non-default model, and test an unknown model

Claude

A live call routed to Gemma-4-12B, and an unknown model comes back as a clear 502 message instead of crashing. Last, a screenshot of the pickers:

Tool calls

- Bash: Screenshot the model pickers and check persistence

User (sent mid-turn)

change the python (uv) env name and flake direnv name to "EvalGen" instead of "hackathon"

Tool calls

- Bash: Find where the env name hackathon is set
- TaskStop: bboah4nhe
- Bash: Rename the flake shell and recreate the venv as EvalGen

Claude

The rename is in place and the tests pass on the new venv. Restarting the server and rerunning the picker check with the wait fixed.

Tool calls

- Bash: Relaunch the EvalGen web app on port 8269
- Bash: Check picker persistence and screenshot the canvas
- Read: `scratchpad/shots/10-pickers.png`

Claude

Model selection is in the web UI, and the env is renamed to EvalGen.

Model pickers

- **Two pickers:** "Model to query" in the Prompt node chooses the model that writes the responses. A picker in the Multi-Evaluator header chooses the model that suggests criteria, writes assertions and grades. This matches the paper's split between pipeline and evaluator models.
- **Model list:** both pickers list what the server at `localhost:11434` offers through a new `/api/models` endpoint. `config.toml`'s `model` is now only the default.
- **Memory:** each picker's choice is saved in the browser. A reload kept GPT-OSS-20B for prompts and Qwen3.6-35B-A3B for evaluation.
- **Mid-run:** the evaluator picker is locked while a run is in progress, so each criterion's assertions and verdicts come from one model. The report card shows which model built each criterion.
- **Navbar:** the old model chip now shows the server address.
- **Checks:** a live criteria call on Gemma-4-12B worked, and an unknown model name comes back as a readable error. All 28 tests pass, along with ruff, black, pyright and the import contract.

Rename

- **Flake:** `flake.nix` now has `description = "EvalGen"` and `DEV_SHELL = "EvalGen"`.
- **Venv:** I recreated `.venv` with `uv venv --prompt EvalGen` and re-synced it. The shell prompt now reads `(EvalGen)`.
- **Package:** the Python package was already named `evalgen`.

The flake name only shows up under `nix develop` or direnv. Both still need `git add flake.nix flake.lock` first, which I've left to you.

The app is running at http://127.0.0.1:8269.